

Jdbc

Jdbc

Jdbc, ovvero Java DataBase Connectivity, è il driver utilizzato dai programmi Java per connettersi ai database relazionali. Sostanzialmente è un insieme di classi, raggruppate principalmente nel package **java.sql**, che consentono di interfacciarsi con un R-DBMS e di lanciare, quindi, comandi SQL da inviare al database relazionale sottostante. Jdbc è indipendente dallo specifico R-DBMS utilizzato, ovvero l'insieme di interfacce e classi che costituiscono Jdbc sono le stesse a prescindere da quale specifico database si vuole utilizzare (Oracle, PostgreSQL, MySql etc.). Esistono, ovviamente, una serie di implementazioni specifiche (librerie .jar) che implementeranno le interfacce di Jdbc per lo specifico database che si decide di usare.

Quindi, dal punto di vista pratico, quando si vuole programmare in Java utilizzando un determinato R-DBMS, come ad esempio faremo nel corso con PostgreSQL, la prima cosa da fare è andare a scaricare la libreria, ovvero il .jar, che implementa Jdbc per quel database.

Ad esempio, nel corso utilizzeremo PostgreSQL versione 9.4, pertanto la prima cosa da fare è scaricare dalla pagina <https://jdbc.postgresql.org/download.html>. In particolare, nel corso siamo interessati alla versione per JDK 8 e pertanto scaricheremo la libreria dall'url: <https://jdbc.postgresql.org/download/postgresql-9.4-1203.jdbc42.jar>.

Una volta scaricata la libreria, per utilizzarla nei nostri codici sorgenti occorre inserirla all'interno del **classpath** dei nostri programmi (cfr. video allegato alla dispensa).



Fatti i due passi appena descritti saremo pronti per iniziare a lavorare con Jdbc nel nostro codice sorgente.

Passiamo quindi a vedere esempi pratici di codice sorgente Java per connettersi ad un database PostgreSQL e per lanciare istruzioni SQL da inviare al database.

La prima cosa da fare nel nostro codice è registrare il driver tramite l'istruzione:

```
Class.forName("org.postgresql.Driver");
```

L'istruzione *Class.forName(String s)* richiede alla JVM di caricare la classe (**class loading**) specificata dal parametro stringa.

Nella fattispecie, come stringa passeremo il nome del driver Jdbc da caricare. A seconda dello specifico R-DBMS che si utilizza cambierà il nome del driver. Generalmente ciascuno fornitore di DBMS (Oracle, Microsoft etc.) fornisce su di una propria pagina web, banalmente ricercabile tramite motori di ricerca come Google o Bing, il nome del driver Jdbc da utilizzare.

Nel caso di PostgreSQL il nome del driver è *"org.postgresql.Driver"*.

A questo punto, nel nostro codice dobbiamo aprire una connessione col database. La connessione al database in Java è rappresentata dall'interfaccia **Connection** del package java.sql.

Per ottenere una connessione, ovvero un'istanza di *Connection*, si utilizza la classe **DriverManager** sempre del package java.sql.

DriverManager espone un metodo **getConnection** overloaded. Noi utilizzeremo la versione che accetta 3 parametri di ingresso:



url del database, utente e password per loggarsi al database.

L'url per connettersi al database è un'altra stringa che varia in funzione dello specifico R-DBMS che si utilizza; anche in questo caso occorre verificare la documentazione fornita dallo specifico produttore del R-DBMS per individuare l'url da usare. Nel caso di PostgreSQL l'url è **`jdbc:postgresql://host:port/database`**, dove occorre sostituire *host* e *port* rispettivamente con l'indirizzo ip e la porta della macchina su cui è installato PostgreSQL e *database* con il nome dello specifico database che vogliamo utilizzare.

Nel mio caso, avendo installato PostgreSQL sulla mia macchina (identificata come **localhost** o come **127.0.0.1**) , sulla porta di 5433 e volendomi connettere al database *corso*, la stringa sarà: `jdbc:postgresql://localhost:5433/corso`.

L'utente e la password sono relativi all'utente che avete creato da interfaccia grafica di amministrazione di PostgreSQL per accedere al database *corso* (cfr. video). Seguendo le istruzioni del video allegato, saranno *username=corso* e *password=corso*.

Ecco il blocco di istruzioni per ottenere una connessione al nostro database *corso*:

```
String dbUrl = "jdbc:postgresql://localhost:5433/corso";
```

```
String userName = "corso";
```

```
String userPassword = "corso";
```

```
connection = DriverManager.getConnection(dbUrl,userName,userPassword);
```



A questo punto siamo pronti per iniziare ad inviare comandi SQL al database, ma prima di farlo occorre definire il concetto di **transazione**.

Una transazione può essere vista come una sequenza di istruzioni SQL che vanno eseguite tutte insieme e che in caso qualcosa vada in errore devono essere tutte annullate. In pratica è un blocco di istruzioni SQL che può essere visto come un'unica istruzione: o va bene tutto o va male tutto. Ci stiamo riferendo quindi alla proprietà di **Atomicità**, del gruppo di proprietà *ACID*, che un R-DBMS deve garantire (cfr. sessione 9-Database).

L'R-DBMS garantisce, quindi, l'impossibilità di trovarci in una situazione di inconsistenza in cui parte del blocco di istruzioni SQL della transazione è stata completata con successo generando modifiche sul database e parte no.

In gergo database si parla di **commit** della transazione quando quest'ultima si chiude con successo e tutte le modifiche apportate dal blocco di istruzioni SQL interne alla transazione vengono scritte sul database.

Viceversa si parla di **rollback** quando un singolo errore all'interno del blocco di istruzioni SQL della transazione determina l'annullamento di tutte le istruzioni SQL della stessa transazione e, pertanto, sul database non viene apportata alcuna modifica; il database, in caso di rollback, rimane nello stesso stato in cui si trovava prima di aprire la transazione.

Per esempio supponiamo di avere nel nostro database una tabella *Conto* in cui manteniamo il saldo del conto corrente dei clienti bancari. Per semplificare supponiamo che la tabella abbia la seguente struttura:



Table Conto (

id bigint NOT NULL,

intestatario character varying(200),

saldo bigint,

CONSTRAINT conto_pkey PRIMARY KEY (id)

)

Il campo *id* è un identificativo univoco del conto (infatti è la primary key della tabella) ed è di tipo *bigint* che in PostgreSQL rappresenta un numero intero che può assumere valori estremamente grandi (giusto per dare un'indicazione: vengono usati 8 byte, quindi il valore massimo è dell'ordine di grandezza degli exabyte ovvero miliardi di miliardi).

Il campo *intestatario* rappresenta il nome dell'intestatario del conto, mentre il campo *saldo* rappresenta il saldo economico del conto.

Supponiamo adesso di voler spostare 1.000€ dal conto identificato dal conto di Mario Rossi (conto n. 1 e saldo iniziale 10.000€) al conto di Carlo Verdi (conto n. 2 e saldo iniziale 5.000€).



In SQL scriveremmo:

```
UPDATE conto SET saldo = 9000 WHERE id=1;
```

```
UPDATE conto SET saldo = 6000 WHERE id=1;
```

Queste due istruzioni **devono** essere inserite in un'unica transazione con commit/rollback alla fine di entrambe.

Tornando al nostro codice, un'istanza di *Connection* appena creata di default si trova in stato di **auto-commit**, ovvero uno stato in cui ogni singola istruzione SQL determina subito un commit o un rollback sul database.

Se lavorassimo in auto-commit, mandando in esecuzione le due istruzioni avremmo un tentativo di scrittura sul database dopo l'esecuzione della prima ed un altro tentativo di scrittura sul db dopo la seconda.

Immaginiamo che la prima vada a buon fine mentre durante l'esecuzione della seconda si verifichi qualche problema, come per esempio una caduta di corrente che determini lo spegnimento del database (anche se in una situazione reale esistono meccanismi di sicurezza che prevengono problemi legati alla caduta di corrente, ma qui stiamo ragionando a titolo di esempio).

Avremmo che il primo conto è stato decurtato di 1.000€ ma il secondo non è stato incrementato della stessa cifra. Ovviamente è una situazione di inconsistenza.



Per evitare che si verifichi una situazione come quella appena descritta occorre eliminare l'auto-commit ed aprire la transazione prima della prima istruzione SQL e chiudere la transazione dopo la seconda.

Tornando al nostro codice dovremo inserire, dopo l'apertura della connessione al database, l'istruzione:

```
connection.setAutoCommit(false);
```

A questo punto per inviare istruzioni SQL ci occorre un'istanza dell'interfaccia **Statement**, sempre del package java.sql.

L'interfaccia *Connection* espone un metodo *createStatement()* che restituisce un'istanza di *Statement*:

```
Statement statement = connection.createStatement();
```

A questo punto, per eseguire istruzioni SQL di scrittura sul database come ad esempio *CREATE TABLE*, *DROP TABLE*, *INSERT INTO*, *UPDATE* etc. possiamo utilizzare il metodo *executeUpdate(String s)* dell'interfaccia *Statement*. Il metodo riceve come parametro di ingresso una stringa che rappresenta l'istruzione SQL che vogliamo mandare in esecuzione.

Ad esempio, per creare una tabella:

```
String createTable = "CREATE TABLE materiacorso (nome character varying(200) NOT NULL,"  
+ "descrizione character varying(50),"
```



```
+ "CONSTRAINT materiacorso_pkey PRIMARY KEY (nome)"  
+ ");
```

```
statement.executeUpdate(createTable);
```

Per inserire un record nella tabella appena creata:

```
String insert = "INSERT INTO materiacorso VALUES ('Corso Java','Corso... ');"
```

```
result = statement.executeUpdate(insert);
```

Per aggiornare un record:

```
String update = "UPDATE materiacorso SET descrizione = 'Corso Java da 0 al web' where nome='Corso Java'";
```

```
result = statement.executeUpdate(insert);
```

Una volta terminato il blocco di istruzioni che vogliamo raggruppare in un'unica transazione occorrerà richiamare il metodo `commit()` dell'interfaccia `Connection` per chiudere la transazione e provare a scrivere sul database.



Il tutto dovrà essere inserito all'interno di un blocco *try/catch*; in caso di errore occorrerà inserire nel blocco *catch* la chiamata al metodo *rollback()* sempre dell'interfaccia *Collection* per annullare gli effetti del blocco di istruzioni SQL presenti all'interno della transazione.

Per eseguire interrogazioni, ovvero **query**, possiamo utilizzare il metodo *executeQuery(String s)* sempre dell'interfaccia *Statement*. Anche per il metodo *executeQuery* il parametro di ingresso è una stringa che rappresenta la query SQL. Il metodo restituisce un'istanza dell'interfaccia **ResultSet**, ovvero un cursore che scorre sull'insieme di record restituiti dalla query (in modo simile a come un *Iterator* scorre sugli elementi di una *Collection* come visto nella sessione dedicata all'utilizzo delle classi di utilità del JDK).

Vediamo subito un esempio:

```
String select = "SELECT * FROM materiacorso";
```

```
ResultSet resultSet = statement.executeQuery(select);
```

```
while ( resultSet.next() )
```

```
{
```

```
    String nome = resultSet.getString("nome");
```



```
String descrizione = resultSet.getString("descrizione");

System.out.println("Nome: " + nome + "\tDescrizione: " + descrizione);

}
```

Notiamo che l'interfaccia *ResultSet* espone il metodo *next()* per scorrere i record restituiti dalla query.

Per ciascun record restituito è possibile accedere al valore dei singoli campi tramite i metodi overloaded *getXXX(String nomeCampo)* e *getXXX(int columnIndex)*:

- *XXX* va sostituito con il tipo di dato che ci aspettiamo per il campo che stiamo leggendo. Nell'esempio mostrato sappiamo che la tabella *MateriaCorso* ha due campi entrambi di tipo stringa di caratteri a lunghezza variabile, ovvero *character varying* in PostgreSQL e *String* in Java, pertanto i metodi da richiamare per leggere i valori dei campi saranno in entrambi i casi *getString("nomeCampo")* oppure *getString(columnIndex)*. Esistono poi i metodi *getInt*, *getBoolean*, *getLong* etc. per gli altri tipi di dati
- si può poi usare una delle due versioni overloaded *getString("nomeCampo")* o *getString(columnIndex)* a seconda se vogliamo accedere al valore del campo attraverso il nome del campo o attraverso la posizione in tabella del campo (a partire da 1 per la prima colonna). Nel caso del nostro esempio potremmo fare riferimento in maniera equivalente a *getString("nome")* o a *getString(1)* e a *getString("descrizione")* o a *getString(2)*

Come al solito, tutto quanto abbiamo appena detto è semplicemente riscontrabile osservando la JavaDoc delle API delle classi e interfacce descritte o direttamente con il supporto di Eclipse (premendo da tastiera “Ctrl + spazio” dopo il carattere “.” dell’istanza su cui vogliamo richiamare i metodi).

Le query SQL inviate al database vengono compilate dal R-DBMS in un linguaggio di basso livello prima di essere effettivamente eseguite. Pertanto, lanciare n volte la stessa istruzione SQL nella quale magari cambino soltanto alcuni parametri non è molto efficiente. Per esempio, immaginiamo di voler eseguire n istruzioni SQL del seguente tipo:

UPDATE materiaCorso SET descrizione='nuova descrizione' WHERE nome='1';

UPDATE materiaCorso SET descrizione='nuova descrizione' WHERE nome='2';

UPDATE materiaCorso SET descrizione='nuova descrizione' WHERE nome='3';

....

UPDATE materiaCorso SET descrizione='nuova descrizione' WHERE nome='n';

In questi casi è più efficiente precompilare un’istruzione SQL parametrica del tipo

UPDATE materiaCorso SET descrizione=? WHERE nome=?;

e mandarla poi in esecuzione sostituendo i parametri ? con i valori effettivi.

Per questo scopo Jdbc mette a disposizione l'interfaccia **PreparedStatement** del package java.sql. Un'istanza di *PreparedStatement* è ottenibile da un'istanza di *Connection* tramite il metodo *prepareStatement(String sql)* al quale va passato come parametro di ingresso la stringa SQL parametrica (quindi con i caratteri "?").

Successivamente si sostituiscono i parametri con i valori effettivi invocando sull'istanza di *PreparedStatement* i metodi *setString(int position,String value)* oppure *setInt(int position,int value)* oppure *setDate(int position,Date value)* e così via per i vari tipi di dati (così come mostrato prima per i metodi *getXXX* dell'interfaccia *ResultSet*). I metodi *setXXX* accettano come parametro di ingresso un *int* che rappresenta la posizione del parametro da sostituire (partendo da 1 per il primo "?", 2 per il secondo "?" e così via) e un oggetto *value* il cui tipo dipende dalla versione del metodo che stiamo usando, ovvero *String* per il metodo *setString*, *Date* per il metodo *setDate*, *int* per il metodo *setInt* e così via.

Infine si manda in esecuzione il codice SQL invocando sull'istanza di *PreparedStatement* il metodo *executeUpdate()*.

Vediamo subito un esempio:

```
String updateParametrico = "UPDATE materiacorso SET descrizione = ? WHERE nome=?";
```

```
PreparedStatement preparedStatement = connection.prepareStatement(updateParametrico);
```

```
preparedStatement.setString(1,"CorsoJava 0->Web");
```

```
preparedStatement.setString(2,"Corso Java");
```

```
preparedStatement.executeUpdate();
```